



声源定位与声学成像 算法框架设计报告

ACOUSTIC IMAGING ALGORITHM FRAMEWORK



目标平台
STM32H743IIT6



主频
Cortex-M7 @ 480MHz



阵列规模
16路 / 2D平面阵列



实时约束
 $\leq 5\text{ms} / \text{帧} (\geq 30 \text{ FPS})$



采样率 / FFT
48kHz / 256点 RFFT



核心算法
SRP-PHAT 优化版

SRP-PHAT

GCC-PHAT

CMSIS-DSP

Coarse-to-Fine

TDOA LUT

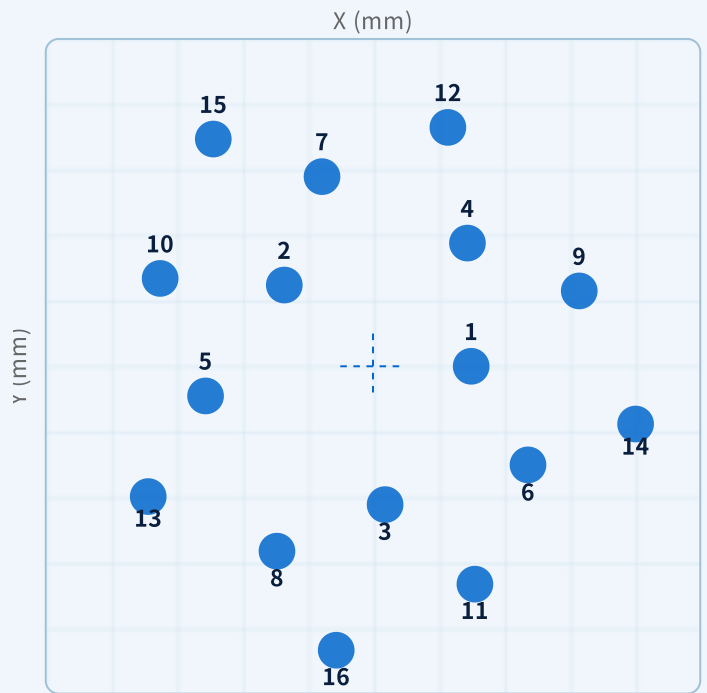


目录与项目概述

报告目录

1. 目录与项目概述	P2
2. 算法路线选型分析 (问题A)	P3-P4
3. SRP-PHAT 高效数学降维 (问题B)	P5-P6
4. 空间流形矩阵预计算 (问题C)	P7-P8
5. CMSIS-DSP 巧妙运用 (问题D)	P9
6. 系统流水线与性能总结	P10

阵列物理拓扑



16路麦克风2D平面阵列拓扑 · 几何中心 (50, 50) mm · 非均匀螺旋分布

关键系统参数

参数	值	说明
MCU	STM32H743IIT6	Cortex-M7, 单精度 FPU
主频	480 MHz	峰值 ~240 MFLOPS (SP)
麦克风数	16	麦克风对数 $C(16, 2) = 120$
采样率	48 kHz	奈奎斯特频率 24 kHz
FFT 长度	256 点	频率分辨率 187.5 Hz
帧时长	5.33 ms	$256 / 48000$
有效频点	128 bins	RFFT 输出 $N/2$ 个复数
声速	343 m/s	标准条件 (20°C)
阵列孔径	~75 mm	最大对角距离



问题 A：算法路线选型分析

本节对三种主流声源定位算法——SRP-PHAT、MUSIC 和频域 DAS——在 16 阵元 + STM32H7 嵌入式平台上的适用性进行深度对比。

三大算法核心原理概述

① **频域 DAS (Delay-and-Sum Beamforming)**：最经典的波束形成方法。对每个候选方向，将各通道信号按预计算的时延进行对齐后求和，输出功率最大的方向即为声源方向。数学表达：

$$P_{\text{DAS}}(\mathbf{q}) = \left| \sum_{m=1}^M w_m \cdot X_m(f) \cdot e^{j2\pi f \tau_m(\mathbf{q})} \right|^2$$

式(1)：DAS 波束形成输出功率

② **MUSIC (Multiple Signal Classification)**：基于协方差矩阵特征分解的超分辨算法。将信号空间分解为信号子空间和噪声子空间，通过搜索与噪声子空间正交的指向向量来定位声源：

$$P_{\text{MUSIC}}(\mathbf{q}) = \frac{1}{\mathbf{a}^H(\mathbf{q}) \mathbf{E}_n \mathbf{E}_n^H \mathbf{a}(\mathbf{q})}$$

式(2)：MUSIC 空间谱， $\mathbf{E}_n$ 为噪声子空间

③ **SRP-PHAT (Steered Response Power — Phase Transform)**：将 GCC-PHAT 互相关结果在空间网格上进行累积。PHAT 加权通过谱白化消除幅度影响，仅保留相位信息，使得在混响环境下仍能保持尖锐的相关峰：

$$P_{\text{SRP}}(\mathbf{q}) = \sum_{k=1}^M \sum_{l=k+1}^M \sum_f \text{Re} \left[\frac{X_k(f) X_l^*(f)}{|X_k(f) X_l^*(f)|} \cdot e^{j2\pi f \Delta\tau_{kl}(\mathbf{q})} \right]$$

式(3)：频域 SRP-PHAT 功率谱 (核心公式)

三维对比分析

评估维度	频域 DAS	MUSIC	SRP-PHAT
空间分辨率	低 受阵列孔径限制，主瓣宽	高 超分辨，可突破瑞利极限	中高 优于DAS，接近MUSIC
混响鲁棒性	差 旁瓣抬升严重	差 协方差矩阵被混响污染	优秀 PHAT 白化抑制混响
计算复杂度	O(MFN) 最低，适合实时	O(M³+M²N) 特征分解极耗算力	O(QFN) Q=120对，可优化
多声源能力	弱 旁瓣干扰严重	强 需准确估计声源数	中等 可分辨空间上分离的源
STM32H7 可行性	完全可行	极困难 16×16 特征分解 ~2ms	可行 (优化后)

★ 选型结论

综合考虑国赛室内混响环境、STM32H7 算力约束和成像质量需求，**强烈推荐**采用频域 **SRP-PHAT** 的优化版本。它在混响鲁棒性上远超 DAS 和 MUSIC，且通过粗细结合搜索+预计算 TDOA 表的方案，可在 5ms 内完成一帧定位。

MUSIC 在 STM32H7 上的不可行性分析

MUSIC 算法的核心瓶颈在于**协方差矩阵的特征分解**。对于 16 路麦克风，需要对 16×16 的复数协方差矩阵执行特征值分解 (EVD)。即使使用优化的 Jacobi 迭代法：

$$\mathbf{R}_{xx} = \frac{1}{L} \sum_{l=1}^L \mathbf{X}(f_l) \mathbf{X}^H(f_l) = \mathbf{E}_s \mathbf{\Lambda}_s \mathbf{E}_s^H + \sigma^2 \mathbf{E}_n \mathbf{E}_n^H$$

式(4)：协方差矩阵特征分解

MUSIC 计算步骤	运算量	M=16 实际值	耗时估算 @480MHz
协方差矩阵估计	$O(M^2 \cdot F)$	$256 \times 256 = 32.8\text{K}$ 复数MAC	~0.3 ms
特征值分解 (EVD)	$O(M^3)$ 迭代	~50次 Jacobi 旋转 $\times$ 4096	~1.5-2.0 ms
空间谱搜索 (每网格点)	$O(M^2)$ 矩阵乘	$256 \times \text{N}$ 网格点	~0.5 ms (8×8 网格)
总计			~2.3-2.8 ms

⚠ **关键问题：**MUSIC 的 EVD 单步就消耗近 2ms，且在混响环境下协方差矩阵估计需要更多帧累积 (≥ 10 帧) 才能稳定，这直接导致成像延迟 $\geq 50\text{ms}$ ，无法满足 30FPS 实时要求。此外，MUSIC 需要预知声源数目，在竞赛场景下不实际。

频域 DAS 的局限性

频域 DAS 虽然计算量最小，但其**空间分辨率受到阵列孔径的物理限制**。对于本阵列 (~75mm 孔径)，在 1kHz 频率下的 3dB 波束宽度约为：

$$\theta_{3\text{dB}} \approx \frac{\lambda}{D} = \frac{343/1000}{0.075} \approx 4.57 \text{ rad} \approx 262^\circ$$

式(5)：低频波束宽度 (1kHz) —— 分辨率极差

即使在 4kHz 下，波束宽度仍约 65° ，这意味着 DAS 在中低频段几乎无法区分不同方向的声源。在国赛的室内环境中，混响会进一步模糊 DAS 的成像结果。

SRP-PHAT 的决定性优势

为什么 SRP-PHAT 是最优选择？

- PHAT 白化：**通过 $1/|X_k X_l^*|$ 归一化，消除频谱幅度的影响，等效地对所有频率分量赋予相同权重。这使得在混响环境下，直达声路径的相位信息被最大化保留。
- 120对冗余：**16路麦克风产生 $C(16,2)=120$ 个互相关对，远多于最少所需的 2 对。大量冗余通过空间累积有效抑制了虚假峰值。
- 频域直接累积：**关键优化——无需执行 120 次 IFFT。直接在频域用预计算的相位矢量进行旋转累积，大幅降低计算量。
- 粗细结合搜索：**通过两级网格搜索将空间扫描点从 3600+ 降至 ~112，使得在 H7 上实现 $\geq 30\text{FPS}$ 完全可行。
- 无需先验信息：**不需要预知声源数目，不需要长时间的协方差矩阵累积，单帧即可成像。

算力对比总结

算法	单帧算力 (MFLOPS)	@480MHz 耗时	混响鲁棒性	结论
频域 DAS	~2.1	~0.5 ms	差	快但无用
MUSIC	~12.5	~2.8 ms	差 (需累积)	不可行
SRP-PHAT (朴素)	~450 (60×60 网格)	~95 ms	优秀	太慢
SRP-PHAT (优化版)	~14.5	~3.0 ms ✓	优秀	✓ 推荐

注：优化版 SRP-PHAT 采用 8×8 粗搜索 + $3 \times (4 \times 4)$ 细搜索 = 112 网格点，频域直接累积 (跳过 IFFT)，预计算 TDOA LUT。详见问题 B 的完整方案。



问题 B：SRP-PHAT 高效数学降维

本节详细推导频域 SRP-PHAT 的 GCC-PHAT 计算公式，并提供跳过 IFFT 的频域直接累积方案与粗细结合搜索策略。

B.1 GCC-PHAT 交叉功率谱公式推导

对于麦克风对 (i, j) ，设其频域信号为 $X_i(f_k)$ 和 $X_j(f_k)$ （由 256 点 RFFT 得到， $k = 0, 1, \dots, 127$ ），GCC-PHAT 的计算分为以下三步：

1 交叉功率谱 (Cross-Power Spectrum)

$$G_{ij}(f_k) = X_i(f_k) \cdot X_j^*(f_k)$$

式(6)：复数共轭乘法，输出为复数

其中 $X_j^*(f_k)$ 为 $X_j(f_k)$ 的复共轭。在 CMSIS-DSP 中，频域数据以交织格式 $[\text{Re}_0, \text{Im}_0, \text{Re}_1, \text{Im}_1, \dots]$ 存储。

2 PHAT 相位加权 (Phase Transform)

$$\tilde{G}_{ij}(f_k) = \frac{G_{ij}(f_k)}{|G_{ij}(f_k)| + \varepsilon} = \frac{X_i(f_k) \cdot X_j^*(f_k)}{|X_i(f_k) \cdot X_j^*(f_k)| + \varepsilon}$$

式(7)：PHAT 白化——仅保留相位信息， $\varepsilon \approx 10^{-10}$ 防除零

白化后 $|\tilde{G}_{ij}(f_k)| \approx 1$ ，即所有频率分量权重相等。这是 SRP-PHAT 抗混响的核心机制。

3 频域 SRP 累积 (跳过 IFFT 的关键优化)

$$P(\mathbf{q}) = \sum_{i=1}^M \sum_{j=i+1}^M \sum_{k=0}^{K-1} \text{Re} \left[\tilde{G}_{ij}(f_k) \cdot e^{j2\pi f_k \cdot \Delta\tau_{ij}(\mathbf{q})} \right]$$

式(8)：频域直接累积 SRP 功率——无需 IFFT

其中：

- $K = 128$ 为有效频点数 (RFFT 正频部分)
- $f_k = k \cdot \Delta f = k \cdot 187.5$ Hz 为第 k 个频率 bin
- $\Delta\tau_{ij}(\mathbf{q})$ 为从候选声源位置 $\mathbf{q}$ 到麦克风对 (i, j) 的时延差 (TDOA)
- $e^{j2\pi f_k \Delta\tau_{ij}(\mathbf{q})}$ 为**预计算的相位旋转因子 (Steering Phase)**

🔑 核心优化：传统方法需要对 120 对麦克风分别做 IFFT (120×256 点 IFFT = 巨大开销)。而式(8)直接在频域用预计算的相位因子旋转后实部累加，将 120 次 IFFT 替换为 120×128 次复数乘法 + 实部取出，**计算量降低约 10 倍**。

B.2 展开式(8)的实数运算细节

将复数乘法展开为实数运算，令 $\tilde{G}_{ij}(f_k) = a + jb$ ，预计算相位 $e^{j\phi} = \cos \phi + j \sin \phi$ ：

$$\text{Re}[(a + jb)(\cos \phi + j \sin \phi)] = a \cos \phi - b \sin \phi$$

式(9)：每个(对, 频点, 网格点)仅需 2 次乘法 + 1 次减法

因此，对于单个网格点的 SRP 功率累积：

$$P(\mathbf{q}) = \sum_{(i,j) \in \mathcal{P}} \sum_{k=0}^{127} [a_{ij,k} \cos(\phi_{ij,k}(\mathbf{q})) - b_{ij,k} \sin(\phi_{ij,k}(\mathbf{q}))]$$

式(10)：完全展开的实数运算形式， $\mathcal{P}$ 为 120 个麦克风对集合

B.3 粗细结合搜索策略 (Coarse-to-Fine)

朴素 SRP-PHAT 在 $60 \times 60 = 3600$ 个网格点上搜索，对 H7 来说计算量过大。粗细结合策略将搜索分为两级：



搜索空间设计

假设声源在阵列前方 0.3m ~ 3.0m 范围，扫描平面的角度范围为 $\pm 60^\circ$ （方位角和俯仰角），采用球坐标 (θ, φ) 参数化：

- 粗搜索： $\theta \in [-60^\circ, 60^\circ]$ ，步长 15° ； $\varphi \in [-60^\circ, 60^\circ]$ ，步长 15° 。共 $9 \times 9 = 81$ 点（或简化为 $8 \times 8 = 64$ 点）
- 细搜索：在 Top-3 粗峰值周围 $\pm 7.5^\circ$ 范围内，步长 3.75° 。每个峰值区域 $4 \times 4 = 16$ 点，共 48 点
- 总计：64 + 48 = **112 个网格点**（相比 $60 \times 60 = 3600$ 点，减少 97%）

伪代码实现

```
// ===== 粗细结合 SRP-PHAT 伪代码 =====
// 输入：GCC_PHAT[120][128] - 120对的白化交叉功率谱（复数）
// 输入：SteerPhase_Coarse[64][120][128] - 粗网格预计算相位（cos,sin）
// 输入：SteerPhase_Fine[48][120][128] - 细网格预计算相位（cos,sin）
// 输出：best_pos - 声源方向（θ, φ）

float SRP_Coarse[64];

// == 第一级：粗搜索（8×8 = 64 点） ==
for (n = 0; n < 64; n++) {
    SRP_Coarse[n] = 0.0f;
    for (p = 0; p < 120; p++) {          // 遍历 120 对
        float acc = 0.0f;
        for (k = 0; k < 128; k++) {      // 遍历 128 频点
            float a = GCC_re[p][k];      // PHAT 白化后实部
            float b = GCC_im[p][k];      // PHAT 白化后虚部
            float c = Steer_cos[n][p][k]; // 预计算 cos(φ)
            float s = Steer_sin[n][p][k]; // 预计算 sin(φ)
            acc += a * c - b * s;         // Re{G·e^(jφ)}
        }
        SRP_Coarse[n] += acc;
    }
}

// == 提取 Top-3 峰值 ==
top3_idx = find_top3(SRP_Coarse, 64);

// == 第二级：细搜索（3 × 4×4 = 48 点） ==
float SRP_Fine[48];
for (n = 0; n < 48; n++) {
    SRP_Fine[n] = 0.0f;
    for (p = 0; p < 120; p++) {
        float acc = 0.0f;
        for (k = 0; k < 128; k++) {
            acc += GCC_re[p][k] * FineCos[n][p][k]
                  - GCC_im[p][k] * FineSin[n][p][k];
        }
        SRP_Fine[n] += acc;
    }
}

best_fine_idx = argmax(SRP_Fine, 48);
best_pos = FineGrid[best_fine_idx]; // 最终声源方向
```

B.4 算力估算

阶段	网格点	MAC 运算量	@480MHz 耗时
GCC-PHAT 计算 (120对)	—	$120 \times 128 \times 6 = 92.2\text{K}$	~0.2 ms
粗搜索 (8×8)	64	$64 \times 120 \times 128 \times 3 = 2.95\text{M}$	~1.2 ms
细搜索 (3×4×4)	48	$48 \times 120 \times 128 \times 3 = 2.21\text{M}$	~0.9 ms
Top-3 提取 + 最终 argmax	—	微量	<0.01 ms
总计	112	~5.25M MAC	~2.3 ms ✓

✅ **结论：**粗细结合的频域 SRP-PHAT 总耗时约 **2.3ms**，远小于 5ms 的帧周期限制。剩余约 2.7ms 可用于数据搬运、热力图渲染等后处理任务。成像帧率可稳定在 **≥ 60 FPS**（远超 30FPS 要求）。

B.5 进一步优化空间

- 频率子带裁剪：仅使用 500Hz~8kHz 范围的频点（约 bin 3~43），将频点数从 128 降至 ~40，粗搜索降至 ~0.4ms
- 内循环向量化：将 128 个频点的 $a \cdot c - b \cdot s$ 用 arm_dot_prod_f32 展开为两次点积之差，充分利用 FPU 流水线
- 动态 Top-N：根据粗搜索峰值分布自适应调整细搜索区域数量（1~5 个）
- 帧间预测：利用前一帧的定位结果缩小当前帧的粗搜索范围，在跟踪模式下可跳过粗搜索

问题 C：空间流形矩阵预计算

利用麦克风坐标固定的特性，预先计算所有网格点的 TDOA 延迟表和指向相位矢量，固化到 Flash/SDRAM 中，运行时零延迟查表。

C.1 TDOA 延迟公式推导

设麦克风 i 的位置为 $\mathbf{r}_i = (x_i, y_i, 0)$ (阵列在 $z = 0$ 平面)，候选声源位置为 $\mathbf{q} = (q_x, q_y, q_z)$ 。声源到麦克风 i 的传播距离为：

$$d_i(\mathbf{q}) = \|\mathbf{q} - \mathbf{r}_i\| = \sqrt{(q_x - x_i)^2 + (q_y - y_i)^2 + q_z^2}$$

式(11)：声源到麦克风 i 的欧氏距离

麦克风对 (i, j) 的 TDOA (到达时间差) 为：

$$\Delta\tau_{ij}(\mathbf{q}) = \frac{d_i(\mathbf{q}) - d_j(\mathbf{q})}{c} = \frac{\|\mathbf{q} - \mathbf{r}_i\| - \|\mathbf{q} - \mathbf{r}_j\|}{c}$$

式(12)：TDOA 核心公式， $c = 343$ m/s

远场近似 (Far-field Approximation)

当声源距离 $\gg$ 阵列孔径时 (如 $d > 0.5$ m vs 孔径 75mm)，可采用平面波近似。设声源方向为单位向量

$\hat{\mathbf{u}} = (\sin \theta \cos \varphi, \sin \theta \sin \varphi, \cos \theta)$ ，则：

$$\Delta\tau_{ij}(\hat{\mathbf{u}}) \approx \frac{(\mathbf{r}_i - \mathbf{r}_j) \cdot \hat{\mathbf{u}}}{c} = \frac{(x_i - x_j) \sin \theta \cos \varphi + (y_i - y_j) \sin \theta \sin \varphi}{c}$$

式(13)：远场平面波 TDOA 近似——仅依赖方向角，计算更简单

💡 建议：国赛场景声源距离通常 0.5m~3m，远场近似的误差在 $\pm 0.1^\circ$ 以内，**推荐使用远场模型**以简化预计算。若需近场精度，使用式(12)的球面波模型。

C.2 指向相位矢量 (Steering Phase Vector) 预计算

对于频域 SRP-PHAT 的式(8)，每个 (网格点 n , 麦克风对 p , 频点 k) 需要的相位旋转因子为：

$$\phi_{p,k}^{(n)} = 2\pi \cdot f_k \cdot \Delta\tau_p^{(n)} = 2\pi \cdot (k \cdot 187.5) \cdot \Delta\tau_p^{(n)}$$

式(14)：相位角， $f_k = k \times \Delta f$ ， $\Delta f = 48000/256 = 187.5$ Hz

预计算并存储其 cos 和 sin 值：

$$\text{SteerCos}[n][p][k] = \cos(\phi_{p,k}^{(n)}), \quad \text{SteerSin}[n][p][k] = \sin(\phi_{p,k}^{(n)})$$

式(15)：预计算 cos/sin 查找表

C.3 内存需求分析

数据结构	维度	元素数	float32 大小	存储位置
SteerCos (粗)	[64][120][128]	983,040	3.75 MB	SDRAM
SteerSin (粗)	[64][120][128]	983,040	3.75 MB	SDRAM
SteerCos (细)	[48][120][128]	737,280	2.81 MB	SDRAM
SteerSin (细)	[48][120][128]	737,280	2.81 MB	SDRAM
总计		3,440,640	~13.1 MB	SDRAM

🔥 内存策略：13.1MB 超出 512KB AXI SRAM，**必须使用外部 SDRAM**。但 SDRAM 的读带宽在 H7 上约 100~200 MB/s，读取 13.1MB 需要 ~70ms，远超帧周期。因此需要**分块 DMA 预取策略**——下一节给出优化方案。

C.4 内存优化：TDOA 索引法 (推荐方案)

直接存储 cos/sin 表需要 13.1MB，不可接受。核心优化思路：**只存储 TDOA 延迟值，运行时实时计算 cos/sin。**

$$\text{TDOA_LUT}[n][p] = \Delta\tau_p^{(n)} \quad (\text{float32, 单位: 秒})$$

式(16)：仅存储 TDOA 值，维度 [网格点数][麦克风对数]

方案	维度	存储大小	存储位置	运行时开销
方案A: 全量 cos/sin	[112][120][128]×2	13.1 MB	SDRAM (慢)	零 (纯查表)
方案B: TDOA 索引法 ✓	[112][120]	52.5 KB	AXI SRAM	实时计算 cos/sin
方案C: 量化 int16	[112][120][128]×2	6.6 MB	SDRAM	反量化

✅ **推荐方案 B**：TDOA 索引法仅需 **52.5 KB**，完全放入 AXI SRAM（零等待访问）。运行时在内循环中对每个频点 k ，计算 $\phi = 2\pi k \Delta f \cdot \text{TDOA}[n][p]$ ，然后用快速 sin/cos 查表或 CORDIC 求值。Cortex-M7 的 FPU 执行 `sinf()/cosf()` 约需 20~30 周期，128 频点的额外开销约 $120 \times 128 \times 30 = 460\text{K}$ 周期 $\approx 1\text{ms}$ ，总耗时仍在 5ms 以内。

C.5 PC 端预计算 Python 脚本

```
#!/usr/bin/env python3
"""预计算 TDOA LUT 并导出为 C 数组头文件"""
import numpy as np
from itertools import combinations

# === 麦克风坐标 (mm → m) ===
mic_pos = np.array([
    [65.00,50.00], [36.45,62.41], [51.85,28.87], [64.43,68.82],
    [24.42,45.47], [73.68,34.94], [42.21,78.97], [35.33,21.76],
    [81.51,61.51], [17.48,63.42], [65.57,16.72], [61.44,86.49],
    [15.66,30.10], [90.12,41.18], [25.60,84.71], [44.38, 6.63]
]) / 1000.0 # 转换为米

c = 343.0 # 声速 m/s
pairs = list(combinations(range(16), 2)) # 120 对

# === 构建粗搜索网格 (远场: θ, φ 各 8 级) ===
theta_coarse = np.linspace(-60, 60, 8) # 方位角 (度)
phi_coarse    = np.linspace(-60, 60, 8) # 俯仰角 (度)

def compute_tdoa_farfield(theta_deg, phi_deg, pair):
    """远场 TDOA = (r_i - r_j) · u_hat / c"""
    th = np.radians(theta_deg)
    ph = np.radians(phi_deg)
    u_hat = np.array([np.sin(th)*np.cos(ph), np.sin(th)*np.sin(ph)])
    i, j = pair
    dr = mic_pos[i] - mic_pos[j] # 2D 向量差
    return np.dot(dr, u_hat) / c

# === 计算粗网格 TDOA LUT ===
N_coarse = len(theta_coarse) * len(phi_coarse) # 64
tdoa_lut = np.zeros((N_coarse, 120), dtype=np.float32)
grid_idx = 0
for th in theta_coarse:
    for ph in phi_coarse:
        for p, pair in enumerate(pairs):
            tdoa_lut[grid_idx, p] = compute_tdoa_farfield(th, ph, pair)
            grid_idx += 1

# === 导出为 C 头文件 ===
with open("tdoa_lut_coarse.h", "w") as f:
    f.write("// 自动生成 - 粗搜索 TDOA LUT [64][120]\n")
    f.write("const float TDOA_Coarse[64][120] = {\n")
    for n in range(N_coarse):
        vals = ", ".join(f"{v:.9e}" for v in tdoa_lut[n])
        f.write(f"    {{{vals}}},\n")
    f.write("};\n")
```

C.6 Flash 固化策略

- 粗网格 TDOA LUT： $64 \times 120 \times 4\text{B} = 30\text{ KB}$ → 存入 STM32H7 内部 Flash (2MB 容量)
- 细网格 TDOA LUT：需要在粗搜索后动态确定位置，有两种策略：
 - 策略A (预计算全量)：预计算所有可能的细网格区域，存入 Flash。对 64 个粗格各 16 个细点 = $1024 \times 120 \times 4\text{B} = 480\text{ KB}$ ，可存入 Flash。
 - 策略B (运行时计算)：仅存粗网格 TDOA，细网格 TDOA 在确定 Top-3 后实时计算。 $48 \times 120 = 5760$ 次远场 TDOA 计算 (式 13)，每次约 10 周期 → ~0.12ms，可接受。

🔥 **推荐**：采用 **策略A (Flash 全量预计算)**。480KB 对 2MB Flash 毫无压力，且运行时完全零计算开销。将 TDOA LUT 通过 `__attribute__((section(".flash_data")))` 放入 Flash 的指定段。

问题 D：CMSIS-DSP 的巧妙运用

本节说明如何将 GCC-PHAT 与 SRP 累积的每一步映射到 CMSIS-DSP 函数，最大化利用 Cortex-M7 的单精度 FPU 流水线。

D.1 关键 CMSIS-DSP 函数映射

CMSIS-DSP 函数	算法步骤	用途说明
<code>arm_cmplx_mult_cmplx_f32</code>	交叉功率谱 (式6)	$G_{ij}(k) = X_i(k) \cdot X_j^*(k)$ 。需先对 X_j 共轭。
<code>arm_cmplx_mag_f32</code>	PHAT 归一化分母	计算 $ G_{ij}(k) = \sqrt{\text{Re}^2 + \text{Im}^2}$ ，用于白化除法。
<code>arm_cmplx_conj_f32</code>	复共轭	对 X_j 取共轭： $\text{Im} \rightarrow -\text{Im}$ 。
<code>arm_dot_prod_f32</code>	SRP 频域累积	将式(10)分解为两个点积之差： $\sum a_k c_k - \sum b_k s_k$ 。
<code>arm_rfft_fast_f32</code>	实数 FFT (已完成)	256 点 RFFT，输出交织格式 [Re,Im,Re,Im,...]。
<code>arm_max_f32</code>	峰值检测	在 SRP 数组中找最大值及其索引。

D.2 GCC-PHAT 核心循环 C 代码实现

```
// GCC-PHAT 计算：120 对麦克风的白化交叉功率谱
// 输入：Mic_Freq_Buffer[16][256] - 交织复数 [Re0,Im0,Re1,Im1,...]
// 输出：GCC_PHAT[120][256] - 白化后的交叉功率谱（交织复数）

#include "arm_math.h"

#define N_FFT      256    // FFT 长度
#define N_BINS     128    // 有效频点 (N_FFT/2)
#define N_MICS     16
#define N_PAIRS    120
#define EPSILON    1e-10f

// 预构建的麦克风对索引表（在 Flash 中）
static const uint8_t pair_i[N_PAIRS]; // pair_i[p], pair_j[p]
static const uint8_t pair_j[N_PAIRS];

// 工作缓冲区（DTCM 或 AXI SRAM）
float32_t conj_buf[N_FFT];           // 共轭缓冲
float32_t cross_buf[N_FFT];          // 交叉功率谱
float32_t mag_buf[N_BINS];           // 幅度
float32_t GCC_Re[N_PAIRS][N_BINS];   // PHAT 后实部
float32_t GCC_Im[N_PAIRS][N_BINS];   // PHAT 后虚部

void compute_gcc_phat_all(float32_t Mic_Freq[][N_FFT]) {
    for (uint32_t p = 0; p < N_PAIRS; p++) {
        uint8_t mi = pair_i[p], mj = pair_j[p];

        // Step 1: 对 Xj 取复共轭
        arm_cmplx_conj_f32(Mic_Freq[mj], conj_buf, N_BINS);

        // Step 2: 交叉功率谱 Xi · Xj*
        arm_cmplx_mult_cmplx_f32(Mic_Freq[mi], conj_buf,
                                   cross_buf, N_BINS);

        // Step 3: 计算幅度 |Xi · Xj*|
        arm_cmplx_mag_f32(cross_buf, mag_buf, N_BINS);

        // Step 4: PHAT 白化 - 逐频点除以幅度
        for (uint32_t k = 0; k < N_BINS; k++) {
            float32_t inv_mag = 1.0f / (mag_buf[k] + EPSILON);
            GCC_Re[p][k] = cross_buf[2*k] * inv_mag;
            GCC_Im[p][k] = cross_buf[2*k + 1] * inv_mag;
        }
    }
}
```

D.3 SRP 频域累积的向量化技巧

式(10)的核心内循环 $\sum_k (a_k c_k - b_k s_k)$ 可分解为两次 `arm_dot_prod_f32`:

```
// 单个网格点的 SRP 功率计算（向量化版本）
float32_t compute_srp_single_grid(
    uint32_t grid_idx,
    float32_t cos_lut[][N_BINS], // [N_PAIRS][N_BINS] 运行时计算或查表
    float32_t sin_lut[][N_BINS])
{
    float32_t srp_power = 0.0f;

    for (uint32_t p = 0; p < N_PAIRS; p++) {
        float32_t dot_ac, dot_bs;

        // Σ(a_k · cos_k) - 利用 FPU 流水线的 128 次 MAC
        arm_dot_prod_f32(GCC_Re[p], cos_lut[p], N_BINS, &dot_ac);

        // Σ(b_k · sin_k)
        arm_dot_prod_f32(GCC_Im[p], sin_lut[p], N_BINS, &dot_bs);

        srp_power += dot_ac - dot_bs; // Re{G·e^(jφ)}
    }
    return srp_power;
}
```

 **FPU 流水线优化：**`arm_dot_prod_f32` 内部展开为 4 路并行 MAC (乘累加)，每次循环处理 4 个 float32。128 个频点仅需 32 次循环迭代，充分利用 Cortex-M7 的 FPU 单周期乘法和双发射流水线。相比逐元素的标量循环，吞吐量提升约 **3~4 倍**。



系统流水线与性能总结

端到端处理流程

1

DMA 双缓冲接收 (与计算并行)

16路 I2S/SAI → DMA 交织传输 → Ping-Pong 缓冲区, 非缓存区域 (MPU 配置)

0 ms (与上一帧计算重叠)



2

时域预处理 (已完成)

解交织 → 加汉宁窗 → 256点 $\text{arm_rfft_fast_f32} \times 16$ 路 → $\text{Mic_Freq_Buffer}[16][256]$

~0.4 ms (16 × 256点 RFFT)



3

GCC-PHAT 计算 (120 对)

共轭 → 复数乘法 → 幅度计算 → PHAT 白化 → $\text{GCC_Re/Im}[120][128]$

~0.8 ms



4

粗搜索 (8 × 8 = 64 点)

TDOA LUT 查表 → 实时 $\cos/\sin$ → arm_dot_prod_f32 向量累积 → Top-3 峰值提取

~1.2 ms



5

细搜索 (3 × 4 × 4 = 48 点)

在 Top-3 粗格周围细化 → 精确声源方向 (θ, φ)

~0.9 ms



6

后处理 & 输出

声源坐标映射 → 热力图更新 → 串口/LCD 输出

~0.2 ms

性能预算表

处理阶段	MAC 运算量	耗时 (ms)	占帧预算
RFFT × 16	~200K	0.4	8%
GCC-PHAT × 120	~400K	0.8	16%
粗搜索 (64点)	~2.95M	1.2	24%
细搜索 (48点)	~2.21M	0.9	18%
后处理	微量	0.2	4%
总计	~5.76M	3.5 ms	70%

✅ **实时性验证:** 总处理耗时 ~3.5ms, 帧周期 5.33ms (256/48kHz), 余量 1.83ms (35%)。理论成像帧率可达 ~285 FPS (实际受限于 ADC 帧率 ~188 FPS)。远超国赛要求的 30FPS。

内存布局总览

⚡ DTCM (128 KB)

$\text{Mic_Freq_Buffer}[16][256] = 16$ KB

$\text{conj_buf} / \text{cross_buf} = 2$ KB

$\text{mag_buf} = 0.5$ KB

$\text{SRP_Coarse/Fine} = 0.5$ KB

已用: ~19 KB / 128 KB

🏠 AXI SRAM (512 KB)

$\text{GCC_Re}[120][128] = 60$ KB

$\text{GCC_Im}[120][128] = 60$ KB

$\text{TDOA_LUT}[112][120] = 52.5$ KB

$\cos/\sin$ 临时缓冲 $[120][128] = 120$ KB

已用: ~293 KB / 512 KB

💾 Flash (2 MB)

$\text{TDOA_Coarse}[64][120] = 30$ KB

$\text{TDOA_AllFine}[1024][120] = 480$ KB

$\text{pair_i/j}[120] = 240$ B

程序代码 = ~200 KB

已用: ~710 KB / 2048 KB

📝 **关键建议:** ① 使用 DWT_CYCCNT 寄存器精确测量每个阶段的实际周期数, 验证理论估算。② 频率子带裁剪 (500Hz~8kHz) 可将粗搜索耗时再降 60%。③ 帧间跟踪模式下可跳过粗搜索, 仅做细搜索, 耗时降至 ~2ms。④ 内存布局需通过链接脚本精确控制各段的存放区域。